

Exercise Sheet 8

Name: Aryan Varshneya

Matriculation Number: 261678

```
In [1]: # Cell 1

import torch
import torch.nn as nn

from torchvision import models, transforms
from torch.utils.data import Dataset, DataLoader

from PIL import Image

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt

from sklearn.metrics import roc_auc_score
from sklearn.neighbors import NearestNeighbors
```

```
In [2]: # Clearing CUDA cache and resetting peak memory stats
torch.cuda.empty_cache()

if torch.cuda.is_available():
    torch.cuda.reset_peak_memory_stats()

print("CUDA cleared")
```

CUDA cleared

```
In [3]: # Cell 2

device = torch.device(
    "cuda" if torch.cuda.is_available()
    else "cpu"
)

print(device)
```

cuda

```
In [4]: # Cell 3

class CarlaDataset(Dataset):

    def __init__(
        self,
        csv_file,
        img_dir,
        label_column=None,
        transform=None
    ):

        self.labels = pd.read_csv(csv_file)
```

```

        self.img_dir = img_dir

        self.label_column = label_column

        self.transform = transform

    def __len__(self):

        return len(self.labels)

    def __getitem__(self, idx):

        row = self.labels.iloc[idx]

        frame = str(row["frame"]).zfill(6)

        img_path = f"{self.img_dir}/{frame}.jpg"

        image = Image.open(
            img_path
        ).convert("RGB")

        if self.transform:

            image = self.transform(image)

        if self.label_column:

            label = float(
                row[self.label_column]
            )

            return image, torch.tensor(label)

        return image

```

```

In [5]: # Cell 4
transform = transforms.Compose([

    transforms.Resize((224,224)),

    transforms.ToTensor(),

    transforms.Normalize(
        mean=[0.485,0.456,0.406],
        std=[0.229,0.224,0.225]
    )
])

```

```

In [6]: # Cell 5
def load_model(model_path):
    model = models.resnet18(
        weights=None
    )
    model.fc = nn.Linear(
        model.fc.in_features,

```

```

        1
    )
    model.load_state_dict(
        torch.load(
            model_path,
            map_location=device
        )
    )
    model = model.to(device)
    model.eval()
    return model

pedestrian_model = load_model(
    "../pedestrian_model.pth"
)
traffic_light_model = load_model(
    "../traffic_light_model.pth"
)
vehicle_model = load_model(
    "../vehicle_model.pth"
)

```

In [7]:

```

# Cell 6
def fgsm_attack(
    image,
    epsilon,
    data_grad
):
    sign_data_grad = data_grad.sign()
    perturbed_image = (
        image +
        epsilon * sign_data_grad
    )
    perturbed_image = torch.clamp(
        perturbed_image,
        -3,
        3
    )
    return perturbed_image

```

In [8]:

```

# Cell 7
def generate_adversarial_example(
    model,
    loader,
    epsilon
):
    model.eval()

    criterion = nn.BCEWithLogitsLoss()

    images, labels = next(iter(loader))

    images = images.to(device)

    labels = labels.to(device).unsqueeze(1)

    images.requires_grad = True

```

```

outputs = model(images)

loss = criterion(
    outputs,
    labels
)

model.zero_grad()

loss.backward()

perturbed_images = fgsm_attack(
    images,
    epsilon,
    images.grad.data
)

return (
    images.detach().cpu(),
    perturbed_images.detach().cpu()
)

```

```

In [9]: # Cell 8
test_dataset = CarlaDataset(
    csv_file="../dataset/test/labels.csv",
    img_dir="../dataset/test/rgb-front",
    label_column="has_traffic_light",
    transform=transform
)

test_loader = DataLoader(
    test_dataset,
    batch_size=32,
    shuffle=False
)

```

```

In [10]: #Cell 9
epsilons = [0.01, 0.05, 0.1]

adv_sets = {}
clean_image = None

for eps in epsilons:

    clean, adv = generate_adversarial_example(
        traffic_light_model,
        test_loader,
        eps
    )

    if clean_image is None:
        clean_image = clean

    adv_sets[eps] = adv

    torch.cuda.empty_cache()

    print(

```

```
        f"Generated epsilon={eps}"
    )
```

Generated epsilon=0.01

Generated epsilon=0.05

Generated epsilon=0.1

```
In [11]: # Cell 10
def denormalize(img):

    mean = torch.tensor(
        [0.485,0.456,0.406]
    ).view(3,1,1)

    std = torch.tensor(
        [0.229,0.224,0.225]
    ).view(3,1,1)

    return img * std + mean
```

```
In [12]: # Cell 11
clean_batch, _ = next(
    iter(test_loader)
)

clean_image = clean_batch[0]

plt.figure(figsize=(12,4))

plt.subplot(1,4,1)

plt.imshow(
    denormalize(clean_image)
    .permute(1,2,0)
)

plt.title("Clean")

plt.axis("off")

for i, eps in enumerate(
    [0.01,0.05,0.1]
):

    adv_img = adv_sets[eps][0].cpu()

    plt.subplot(1,4,i+2)

    img = denormalize(adv_img)
    img = torch.clamp(img, 0, 1)

    plt.imshow(
        img.permute(1,2,0)
    )

    plt.title(
        f"Eps={eps}"
    )
```

```
plt.axis("off")

plt.show()
```

Clean



Eps=0.01



Eps=0.05



Eps=0.1



Excercise 8.5

```
In [13]: # Cell 12
from sklearn.metrics import recall_score
```

```
In [16]: # Cell 13
def evaluate_adversarial_recall(
    model,
    loader,
    epsilon
):
    model.eval()

    criterion = nn.BCEWithLogitsLoss()

    all_preds = []
    all_labels = []

    for images, labels in loader:
        images = images.to(device)

        labels_gpu = (
            labels
            .to(device)
            .unsqueeze(1)
        )

        images.requires_grad = True

        outputs = model(images)

        loss = criterion(
            outputs,
            labels_gpu
        )

        model.zero_grad()

        loss.backward()

        perturbed = fgsm_attack(
            images,
```

```

        epsilon,
        images.grad.data
    )

    outputs = model(
        perturbed
    )

    preds = (
        torch.sigmoid(outputs)
        > 0.5
    ).float()

    all_preds.extend(
        preds.cpu().numpy()
    )

    all_labels.extend(
        labels.numpy()
    )

    recall = recall_score(
        all_labels,
        all_preds
    )

    return recall

```

```

In [17]: # Cell 14
clean_recall = 0.0

for eps in [
    0.01,
    0.05,
    0.1
]:

    adv_recall = (
        evaluate_adversarial_recall(
            traffic_light_model,
            test_loader,
            eps
        )
    )

    print(
        f"Epsilon={eps}"
    )

    print(
        f"Recall={adv_recall:.4f}"
    )

```

```

Epsilon=0.01
Recall=0.9195
Epsilon=0.05
Recall=0.0774
Epsilon=0.1
Recall=0.0004

```

```
In [18]: # Cell 15
def evaluate_clean_recall(
    model,
    loader
):

    model.eval()

    all_preds = []
    all_labels = []

    with torch.no_grad():

        for images, labels in loader:

            images = images.to(device)

            outputs = model(images)

            preds = (
                torch.sigmoid(outputs)
                > 0.5
            ).float()

            all_preds.extend(
                preds.cpu().numpy()
            )

            all_labels.extend(
                labels.numpy()
            )

    return recall_score(
        all_labels,
        all_preds
    )
```

```
In [19]: # Cell 16
clean_recall = evaluate_clean_recall(
    traffic_light_model,
    test_loader
)

print(
    f"Clean Recall = {clean_recall:.4f}"
)
```

Clean Recall = 0.9834

```
In [20]: # Cell 17
results = []

for eps in [0.01,0.05,0.1]:

    adv_recall = evaluate_adversarial_recall(
        traffic_light_model,
        test_loader,
        eps
    )
```



```

recall_drop = (
    clean_recall -
    adv_recall
)

results.append([
    eps,
    clean_recall,
    adv_recall,
    recall_drop
])

results_df = pd.DataFrame(
    results,
    columns=[
        "Epsilon",
        "Clean Recall",
        "Adversarial Recall",
        "Recall Drop"
    ]
)

results_df

```

Out[20]:

	Epsilon	Clean Recall	Adversarial Recall	Recall Drop
0	0.01	0.983359	0.919505	0.063854
1	0.05	0.983359	0.077399	0.905960
2	0.10	0.983359	0.000387	0.982972

In [27]: *# Cell 18*

```

pedestrian_dataset = CarlaDataset(
    csv_file="../dataset/test/labels.csv",
    img_dir="../dataset/test/rgb-front",
    label_column="has_pedestrian",
    transform=transform
)

traffic_dataset = CarlaDataset(
    csv_file="../dataset/test/labels.csv",
    img_dir="../dataset/test/rgb-front",
    label_column="has_traffic_light",
    transform=transform
)

vehicle_dataset = CarlaDataset(
    csv_file="../dataset/test/labels.csv",
    img_dir="../dataset/test/rgb-front",
    label_column="has_vehicle",
    transform=transform
)

```

In []: *## Cell 18.1*

```

# from torch.utils.data import Subset
# import numpy as np

```

```

# np.random.seed(42)

# indices = np.random.choice(
#     len(pedestrian_dataset),
#     100,
#     replace=False
# )

# pedestrian_subset = Subset(
#     pedestrian_dataset,
#     indices
# )

# traffic_subset = Subset(
#     traffic_dataset,
#     indices
# )

# vehicle_subset = Subset(
#     vehicle_dataset,
#     indices
# )

```

```

In [32]: pedestrian_loader = DataLoader(
    pedestrian_dataset,
    batch_size=4,
    shuffle=False
)

traffic_loader = DataLoader(
    traffic_dataset,
    batch_size=4,
    shuffle=False
)

vehicle_loader = DataLoader(
    vehicle_dataset,
    batch_size=4,
    shuffle=False
)

```

```

In [33]: # Cell 20
models_dict = {
    "Pedestrian": (
        pedestrian_model,
        pedestrian_loader
    ),
    "Traffic Light": (
        traffic_light_model,
        traffic_loader
    ),
    "Vehicle": (
        vehicle_model,
        vehicle_loader
    )
}

```

```

In [30]: torch.cuda.empty_cache()

```

```
In [34]: # Cell 21
all_results = []

for model_name, (
    model,
    loader
) in models_dict.items():

    clean_recall = evaluate_clean_recall(
        model,
        loader
    )

    for eps in [0.01, 0.05, 0.1]:

        adv_recall = evaluate_adversarial_recall(
            model,
            loader,
            eps
        )

        recall_drop = (
            clean_recall -
            adv_recall
        )

        all_results.append([
            model_name,
            eps,
            clean_recall,
            adv_recall,
            recall_drop
        ])

results_df = pd.DataFrame(
    all_results,
    columns=[
        "Model",
        "Epsilon",
        "Clean Recall",
        "Adversarial Recall",
        "Recall Drop"
    ]
)

results_df
```

Out[34]:

	Model	Epsilon	Clean Recall	Adversarial Recall	Recall Drop
0	Pedestrian	0.01	0.277620	0.118980	0.158640
1	Pedestrian	0.05	0.277620	0.096317	0.181303
2	Pedestrian	0.10	0.277620	0.232295	0.045326
3	Traffic Light	0.01	0.983359	0.919118	0.064241
4	Traffic Light	0.05	0.983359	0.077012	0.906347
5	Traffic Light	0.10	0.983359	0.000387	0.982972
6	Vehicle	0.01	0.877407	0.755556	0.121852
7	Vehicle	0.05	0.877407	0.713333	0.164074
8	Vehicle	0.10	0.877407	0.873704	0.003704

In [35]:

```
# Cell 22
results_df.pivot(
    index="Epsilon",
    columns="Model",
    values="Recall Drop"
).plot(
    kind="bar",
    figsize=(8,5)
)

plt.ylabel("Recall Drop")
plt.title("FGSM Robustness Evaluation")
plt.show()
```

